



From Demo to Production: A Year of Agentic AI in Financial Markets

Anton Antonov (he/him)

Head of AI and Quantitative Research

dxFeed

Arseni Yustus (he/him)

Solutions Architect

Amazon Web Services



Obstacles on the path to production

Excitement
and potential



POC

Challenges on the path to production



Acceptance
Criteria



Hidden
Gaps



Use-Case
Sizing



Enterprise
Integration



Security &
Compliance

Meaningful
business value



Production



A demo proves it's possible. Production proves it holds up.

DEMO / POC

Built to demonstrate capability

- Low volume, clean inputs
- A patient tester - usually you
- Negligible bill, looks fast

PRODUCTION

Built to survive real load

- Business volume, messy inputs
- Impatient users, zero tolerance
- Cost, latency & failures surface at scale

Invisible in a demo:

Cost

Latency

Reliability

Failure modes



Evals As Acceptance Criteria

An eval is a structured test that checks whether your system returns the expected, accurate output. Writing the suite before production code forces three things to happen.



Define success

State what success means in measurable terms - before opinions creep in.



Expose assumptions

Surface design assumptions early, while changing them is still cheap.



Create a change-gate

Know whether a model swap, prompt change, or new retrieval actually improved things.

Defining success criteria

“Summarize claims accurately” doesn’t tell you what to measure. Turn it into a measurable threshold in four steps.

1

Identify the behavior

Be specific: "extract the filer's name, claim number, incident date, and claimed amount."

2

Set the threshold

100% on structured fields, <2% hallucination, 99.5% schema-valid — from the requirement, not the prototype.

3

Identify failure modes

A fake claim number, a missing date, a value from the wrong claim. Each is a category in your dataset.

4

Include adversarial inputs

Missing fields, handwriting, unusual formats. A clean-only golden set won't predict production.

Evaluating models, knowledge-bases and agents

Programmatic/LLM evaluation



Accuracy



Robustness



Toxicity



Correctness



Completeness



Helpfulness



Creativity



Style



Tone



Relevance



Coherence



Readability



Accuracy



Consistency



Brand voice

Retrieval & Generation



Context Coverage



Context Relevance



Correctness



Completeness



Helpfulness



Citation precision



Citation coverage



Logical coherence



Faithfulness

Responsible AI



Harmfulness



Stereotyping



Refusal

Custom metrics

Three ways of evaluation

Each type trades speed and cost against flexibility. Use the cheapest one that can judge the behavior reliably.

FASTEST · CHEAPEST

Code-based

Deterministic checks that run in milliseconds and cost almost nothing.

FLEXIBLE · MODERATE

Model-based

A judge model assesses outputs that need interpretation - roughly one model call.

SLOWEST · PRICIEST

Human-review

Human judgment for high-stakes or novel behavior neither code nor a model can be trusted on.



Rule of thumb: code → model → human runs slower and pricier at each step. Push checks as far left as they'll reliably go.

Model the cost before you write code



Cost

Negligible at 10–50 req/day; at scale the bill can blow past the budget signed off at approval.



Latency

Fine for one patient tester; too slow under real, concurrent load.



Reliability

Production demands retries, fallbacks, and circuit breakers.



Failure modes

Name each one for the chosen architecture and articulate an explicit mitigation.



Capture the first business-metric signal on your POC sample — fitting the budget but not moving the metric is still a failed deployment.

Four dimensions where a demo misleads you

Sizing means producing a cost model accurate enough to validate the architecture against the budget. Four inputs drive it.



1

Call volume

Requests per day / month
- from the business owner,
not developer intuition.



2

Token budget

Tokens per request, input
plus output.



3

Model tier

Which model the use case
actually requires.



4

Sensitivity

The parameters that swing
the estimate most.



Example: 1,000 conversations/day = 1,000 calls/day plus multi-turn continuations. A sample dataset won't give an accurate volume figure.

Reliability is architecture, not the model

Output quality is validated through evals; system reliability is validated through architecture controls — the patterns that keep a system standing under real load.



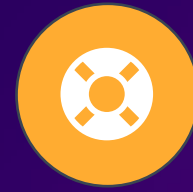
Retries

Re-attempt transient failures - with backoff, so you don't amplify an outage.



Fallbacks

Degrade gracefully to a simpler model, a cached result, or a safe default.



Circuit breakers

Stop calling a failing dependency, fail fast, and give it room to recover.



Agents need these most. Systems that use tools and reason across turns have more steps - and more places to fail.

Five layers, five architect decisions

1



Entry point

Decided first — compliance eliminates options here

2



Integration pattern

How the system connects and exchanges data

3



Identity

SSO / OAuth — authentication of callers

4



Authorization

What an authenticated caller is allowed to do

5



Data handling & residency

Where data is processed and stored

Constraints shape the design first

Regulatory and policy constraints eliminate entry-point options before any other decision. Match the integration pattern to the constraint.



Data privacy

HIPAA, GDPR - BAA coverage and controls on personal / health data.



Data residency

Where data is processed and stored; FedRAMP and jurisdiction limits.



Identity

SSO / OAuth - who is allowed to invoke the system.



Authorization

Privilege via contractual terms, retention & internal policy.

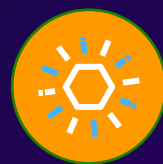
Not legal guidance — work with your own legal and compliance team to implement controls that meet your organization's needs.

"Ready" means clearing two bars at once



Output quality

Validated through evals.



System reliability

Retries, fallbacks, circuit breakers.

THE PATH TO PRODUCTION

1

Define success and build the eval suite first

2

Model cost and reliability before you deploy

3

Verify feasibility before you commit

4

Design an integration that survives security review

About dxFeed

dxFeed is a subsidiary of [Devexperts](#), with the primary focus of delivering financial markets information to buy-side and sell-side institutions and individuals, specifically to traders, data analysts, quants and portfolio managers.

6M

End-Users through
direct and B2B2C
relationships

200,000

Simultaneous streaming clients

3.5M

Instruments
(Equities, Futures,
Options, Indices, FX,
Crypto, etc.)

>100

B2B clients around
the world



What we shipped this year



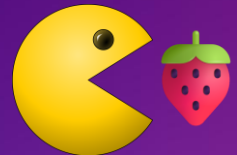
Product	Headline
Grenadier-2	Capable of streaming full US equity universe @ 4,000 RPS; production latency p50 51 ms
GDELT News -> SummerFox	Licence-free news source, live in SummerFox; includes stocks, commodities, crypto
dxMCP	Market data + Grenadier + SummerFox on one MCP surface

The model stopped being the interesting part

Everything that makes AI work in production now lives in the harness around it.

Three proofs from one year of shipping:

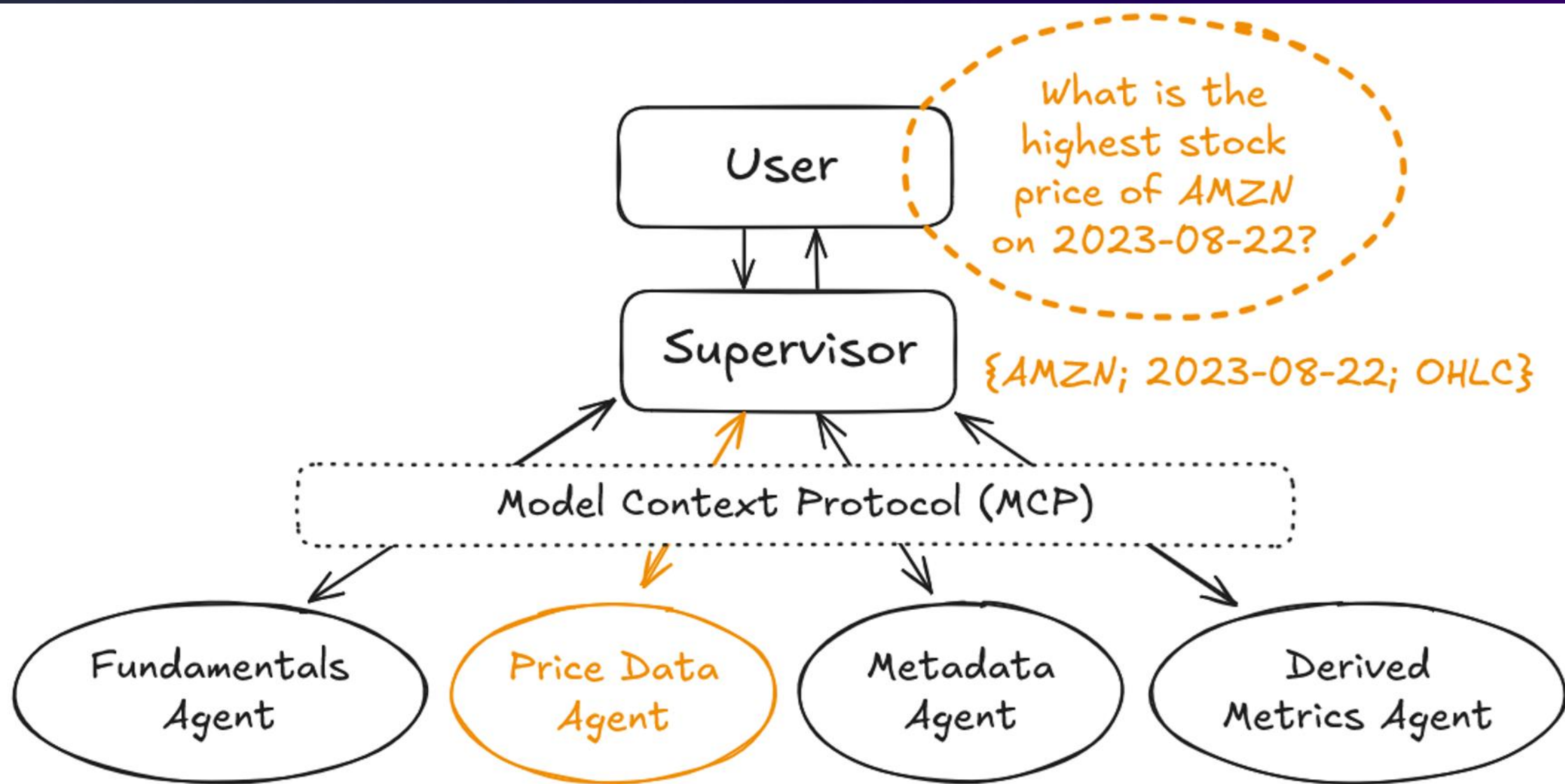
1. a gateway that turns products into tools,
2. a pipeline that buys model judgement by the call,
3. and a product with no LLM in it at all.

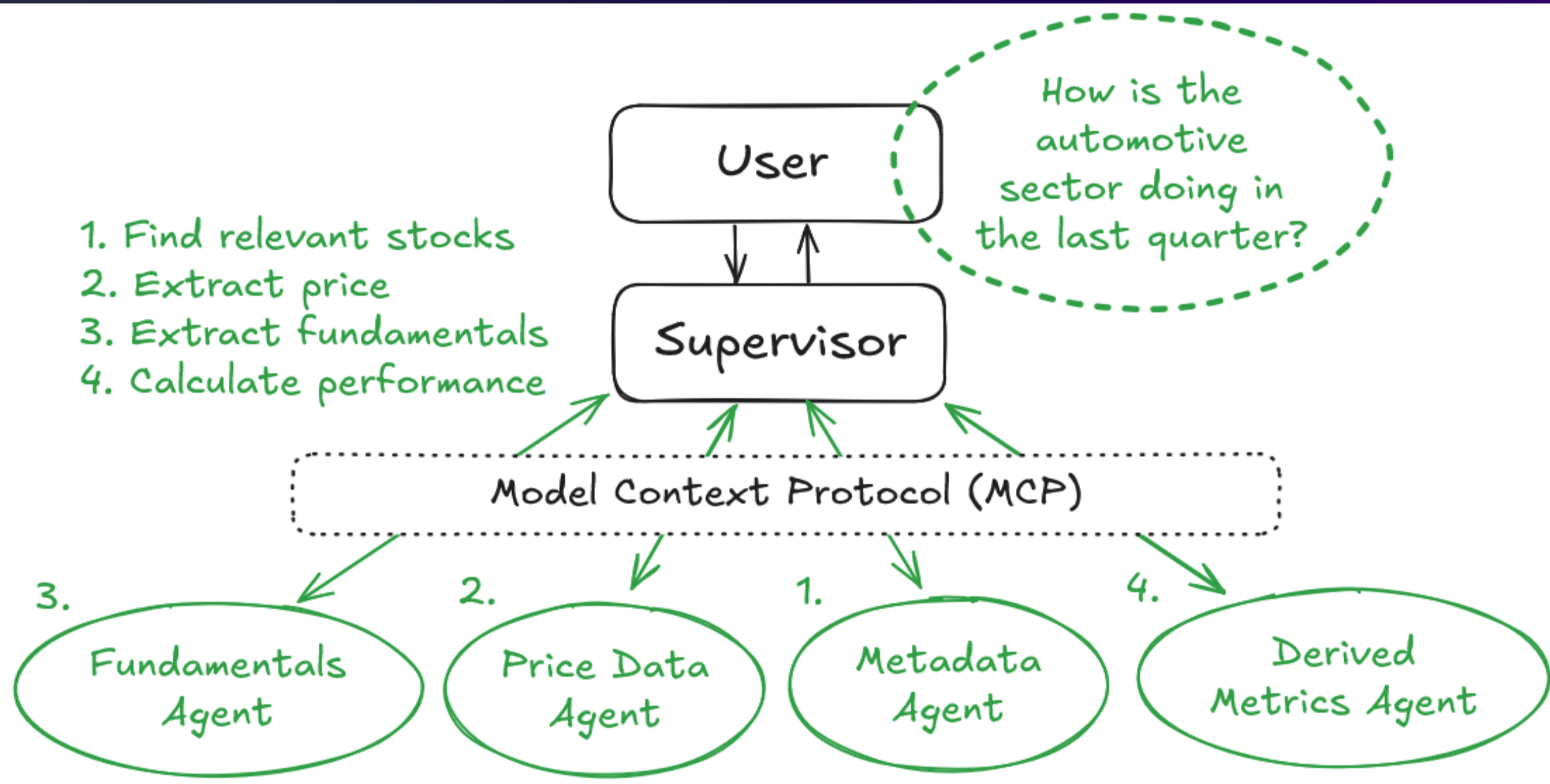


dxMCP: what's on the surface

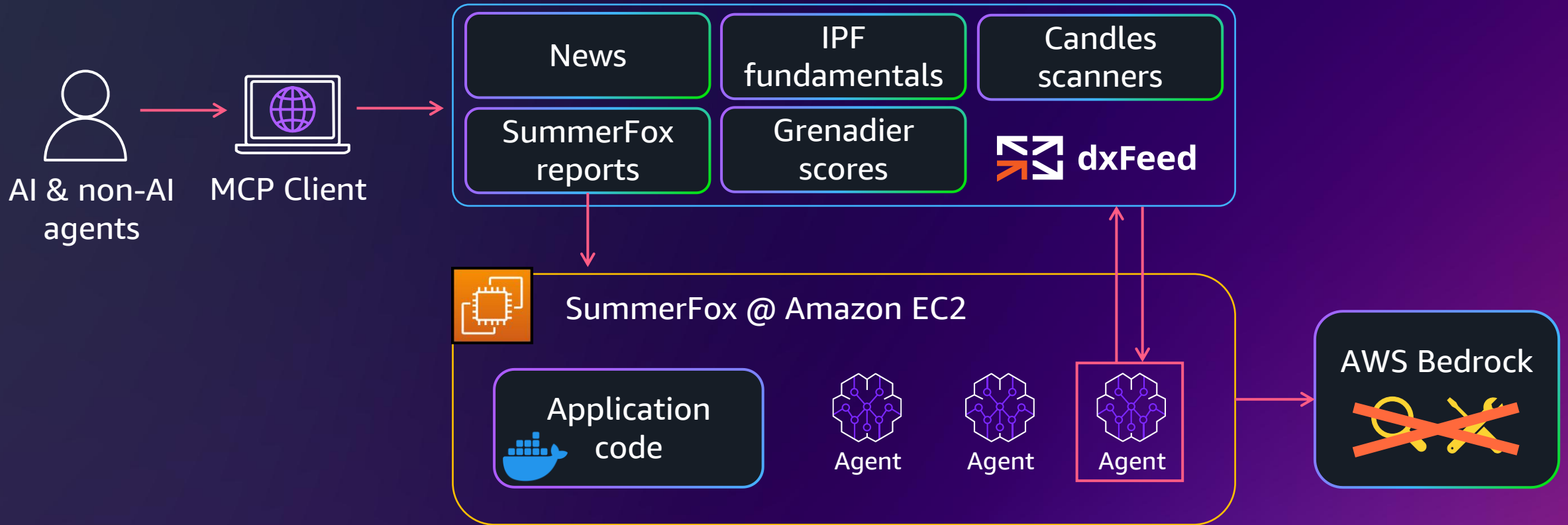
Model Context Protocol: the open standard for connecting AI applications to external systems.

Layer	On the surface today
Market data	Candles / OHLC, trades & quotes, instrument profiles (IPF), fundamentals, news
Screening	Options and underlying scanners - 400+ datapoints
Products as tools	Grenadier anomaly scores, SummerFox per-symbol and portfolio summaries
Consumers	AI agents and human operators, e.g. any MCP client on a desktop Products: DXcharts, DXtrade AI agent, SummerFox





Products as tools, not just apps



Decision rule #1: when MCP is the right answer

Use it when

The answer already exists somewhere authoritative

The model just needs to fetch it correctly

More than one consumer will want the same data

Don't use

As a substitute for a calculation engine

When the consumer is one fixed pipeline – just call the API

To deal with a data-quality problem upstream

SummerFox: the market report, written by agents

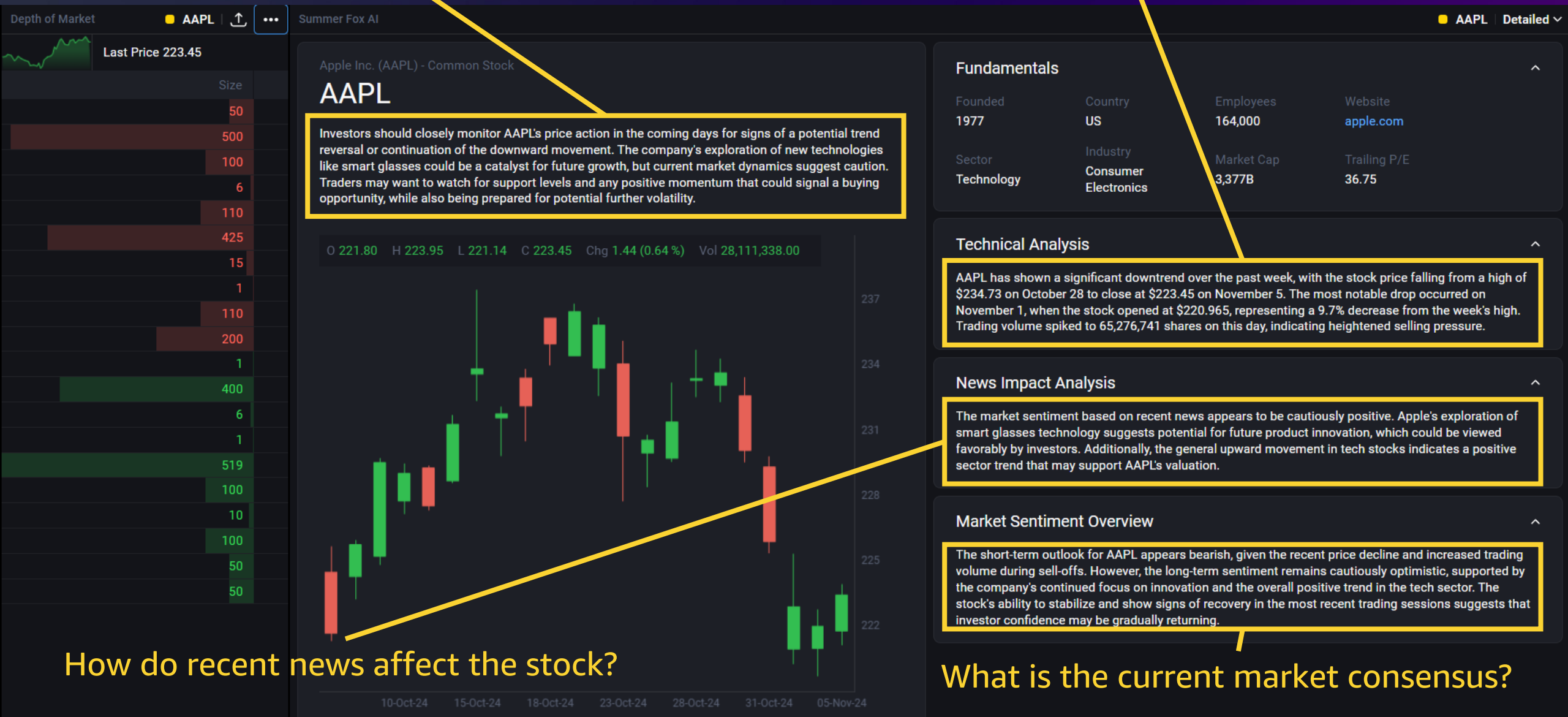
- Ask about a stock or a portfolio, get a structured report in seconds
- Every number & data point comes from dxFeed: candles, news, fundamentals, instrument profiles
- One report, any language
- Per-symbol and portfolio views
- Now with licence-free financial news on Google's GDELT GKG (BigQuery public dataset)



dxFeed SummerFox in Action

What is the high-level outlook?

What data-driven insights are there?



A headline is not a ticker

What the text says	The trap	What gets stored
"Marvell to join S&P 500"	The louder name is not the subject	stock:MRVL primary index:SPX secondary
"[...] Source: Nasdaq", "A quiet day on Nasdaq"	The org name produces bogus NDAQ matches	nothing
"COMP rises 2%" with no class prefix	Compass Inc. or the Nasdaq Composite? The string cannot tell you	nothing, not guessed

Turning a sentence into a symbol is a judgement – exactly where a model is both necessary and dangerous.

Still no silver bullet



Text volume: ~330 tokens / article	LLM cost: \$0.4 – \$8 per Mtok	Runtime: 6 s/call	Precision (article -> ticker)
All articles: ~49 K/day	\$6.5 – \$130/day	~8 hrs/day	27%

Four layers, each with one job



Text volume: ~330 tokens / article	LLM cost: \$0.4 – \$8 per Mtok	Runtime: 6 s/call	Precision (article -> ticker)
All articles: ~49 K/day	\$6.5 – \$130/day	~8 hrs/day	27%
Gated approach: 1350/day	<\$4/day	14 min/day	68%

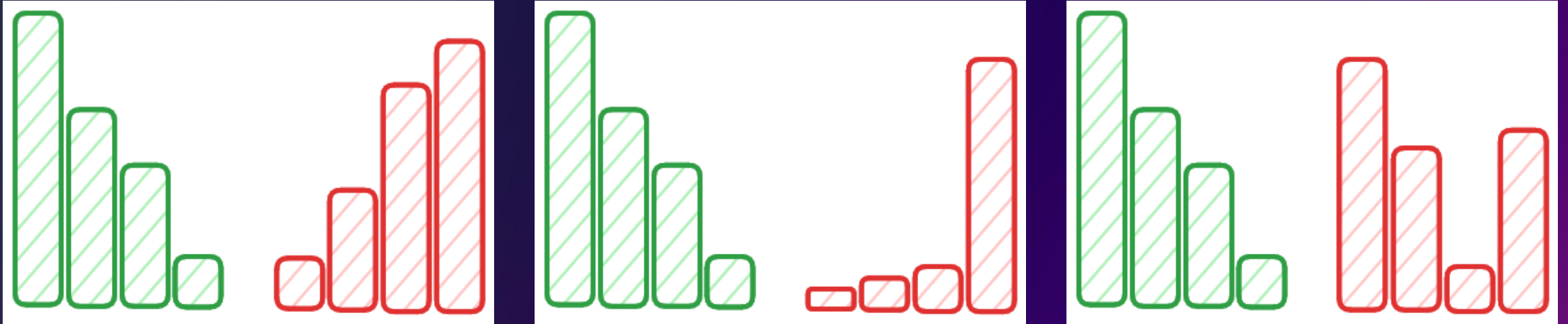
Decision rule #2: where the model belongs

1. Put the model where judgement is genuinely required.
2. Put deterministic code on both sides of it.
3. Make something other than the model decide what gets stored.

This results in predictable costs, runtime, and precision.

Sometimes the harness contains no model at all

Grenadier: a 0-100 anomaly score for the order book of every US-listed stock, once a second, from one in-house model. No LLM anywhere in it!

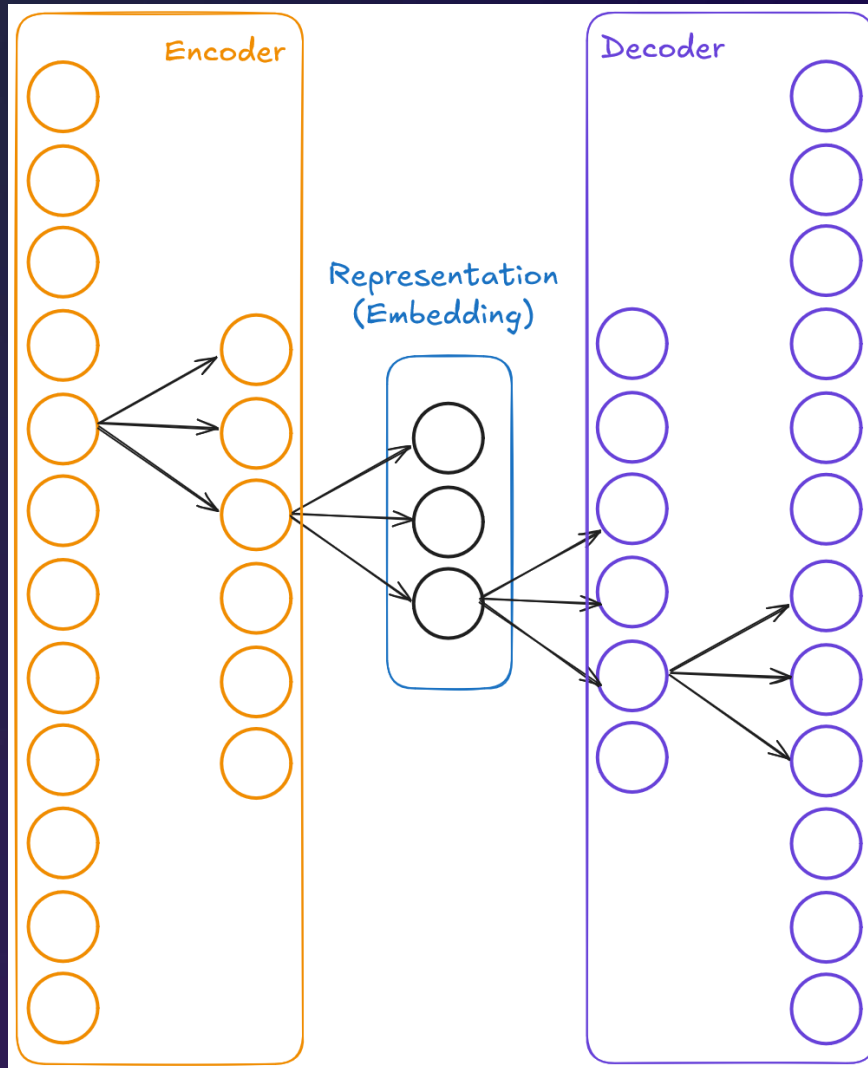
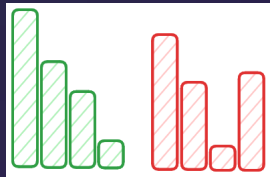
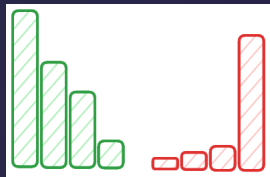
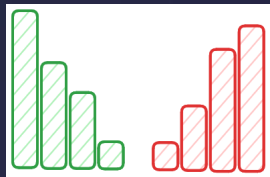


The needle in the haystack problem: rare and fuzzily defined events

Rule-based or feature engineered conditions:

- must be manually constructed or selected
- must be calibrated and recalibrated, may get outdated quickly

The bet: one universal model, or a zoo



The key risk:

"Transfer learning fails: a universal model that covers a wide instrument universe is unfeasible."

Fallback plan: a per-symbol model zoo.

- One unsupervised autoencoder, 20 GB of proprietary L2 order-book data
- Generalises to new instruments without retraining

Target vs delivered

	Target	Delivered
Coverage	2,600+ US stocks	Full US equity universe
Throughput	2,600 RPS (1/symbol/sec)	4,000 RPS
Latency (same region)	Sub-second	p50 51 ms p99 223 ms
Running cost	Fixed budget per month	~1/5 of the budget ~1/20 for a representative subset

dxFeed Grenadier in Action

Large volume concentration far from the spread



Decision rule #3: when to train your own model

	Use an LLM when... (SummerFox)	Train your own when... (Grenadier)
The input	is language: headlines, filings, a question a human typed	is a numeric signal: 20 GB of order-book snapshots
"Correct" means	a judgement a human expert would agree with	a statistic: reconstruction error against what normal looks like
The knowledge	lives in the world — companies, events, context	lives only in your data — there's no LLM that ever saw it
Volume & latency	thousands of calls a day, seconds are fine	thousands of inferences a second, forever, with sub-second latency — no LLM does that at any price
Economics	pay per call because each call is worth it	pay per day/month because load is tested and predictable

Six evals, three buckets

Type	Our layer	What it catches
Code	Data: assertions vs source of truth	Wrong values
Code	Tool: trace replay against golden traces (SummerFox)	Right answer, wrong call
Code	Config: label menus test-pinned to the prompt	Config drift
Code	Model: precision/recall on a held-out labelled set (Grenadier)	Detection quality
Model-graded	Agent: faithfulness, no fabrication (Inspect, Deepeval)	Plausible fiction
Human	Sampled review of production traces, back into the golden set	Everything we didn't think of

Bonus: quality control and AI economics

Noise floor: ~40% of articles come back different on identical input

Each prompt edit, no matter how trivial, is a new deployment

Multi-vendor fallback helps deal with outages and model degradations

No reproducibility: measure old-vs-old before you measure old-vs-new

Approach your AI-related costs on a per-call, not per-month, basis

Token accounting must be built into the audit trail

The blueprint

Does the answer already exist somewhere authoritative?



MCP *fetch it correctly; never compute it*

Is judgement genuinely required?



A model – gated approach *deterministic code on both sides, something else decides what is stored*

Narrow task, high volume, your own data?



Train your own model *no LLM does 2,600 inferences a second at any price*

**Key call: establish key metrics and estimate design decisions
before you start scaling.**

Key Takeaways

- The model is a component. The harness is the product.
- Provide facts as tools via MCP.
- Rely on LLM judgement in a gated model.
- Form numeric, high-volume, high-throughput signals in your own specialised model.
- Price and measure every call before you pay for it, or the pilot never becomes a product.
- Sometimes the right harness contains no LLM at all.

dxFeed SummerFox: AI-driven market reporting



<https://dxfeed.com/solutions/summerfox-market-intelligence-plus-ai/>

dxFeed Grenadier: L2-based anomaly detection

dxFeed Grenadier —
Best AI-based
Solution

at Waterstechnology Asia Awards
2026



<https://dxfeed.com/solutions/dxfeed-grenadier/>

Press release: <https://dxfeed.com/dxfeed-grenadier-recognized-as-best-ai-based-solution/>



Thank you!

Anton Antonov

✉ antonov@dxfeed.com
in [linkedin.com/in/tonytonov](https://www.linkedin.com/in/tonytonov)

Arseni Yustus

✉ ayustus@amazon.com
in [linkedin.com/in/arseni-yustus](https://www.linkedin.com/in/arseni-yustus)